IDC409 Finance + News

Getting started guide: from a fresh computer to 180 days of real data, one step at a time

You do **not** need Python, Git or any programming knowledge. This guide starts from zero: install Python, download the project from GitHub, collect 180 days of market and news data, then use each feature of the project separately.

Part	What you do	Time
1	Install Python and download the project (one time)	15 min
2	Check everything works using fake demo data	2 min
3	Collect 180 days of real data, step A, B, C	15 min
4	Use each feature one by one (menu of commands)	as needed
5	Run the notebooks and read the results	10 min
6	Fixing common problems	

You need: a Windows, Mac or Linux computer, an internet connection, and about 1 GB of free disk space. Everything in this guide is free.

Part 1 · Install and download (one time only)

Step 1.1 · Install Python

Go to **python.org/downloads**, download the newest Python 3 (version 3.10 or higher) and run the installer.

Windows: on the first installer screen, tick the box **"Add python.exe to PATH"** before clicking Install. This is the most common mistake, and it is why people later see "python is not recognized".

Check it worked: open a terminal (Windows: press the Start key, type **cmd**, press Enter. Mac: open the **Terminal** app) and type:

```
python --version          (Mac/Linux: python3 --version)
```

You should see something like *Python 3.12.4*.

Step 1.2 · Download the project from GitHub

Open the project's GitHub page, click the green **Code** button, then **Download ZIP**. Unzip it (right-click, Extract All). You get a folder called **idc409-finance-news**. Put it somewhere easy such as your Desktop. (If you already use Git you can *git clone* instead.)

Step 1.3 · Open a terminal inside that folder

Windows: open the folder in File Explorer, click the address bar at the top, type **cmd** and press Enter. **Mac:** right-click the folder and choose *New Terminal at Folder*. Every command in this guide is typed in this terminal, followed by Enter.

Step 1.4 · Create the project's private environment and install the libraries

Windows:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

Mac / Linux:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

The install takes 3 to 10 minutes. When it finishes you see **(.venv)** at the start of the line.

> **Every time you open a new terminal** you must activate the environment again (the second command above) before running anything. If you forget, you will see "No module named ..." errors.

Part 2 · Check it works with demo data

This uses **fake (synthetic)** data generated on your computer, so it needs no internet. It only proves the installation is correct.

```
python run_pipeline.py --demo
python tests/test_pipeline.py
```

Expected last line of the second command: **8 tests passed**. If you see that, you are ready for real data.

Part 3 · Collect 180 days of real data

Run the three steps below in order. Doing them one by one lets you see exactly what each stage produces.

> **Why `--years 0.5` with `--news-days 180`?** Prices must cover the same period as the news. If prices go back further than the news, the older days would have zero headlines and the models would learn from missing news. 0.5 years is about 182 days.

Step A · Download prices (about 1 minute)

```
python run_pipeline.py --steps prices --years 0.5
```

You will see one line per series (Nifty, Sensex, shares, gold, petrol, Brent, USD/INR, EUR/INR) with a row count. Each should show roughly 120 to 185 rows. A row count of 0 means that series failed (see Part 6).

Step B · Download news headlines (about 10 minutes)

```
python run_pipeline.py --steps news --news-days 180 --delay 0.7
```

The program fetches one search per day and topic, politely, about one per second, and prints progress every 50. **You can stop at any time with Ctrl+C and run the same command again; it skips days already finished.** If the network drops, it stops after 8 failures in a row; simply run it again later.

Step C · Build the dataset (a few seconds)

```
python run_pipeline.py --steps build
```

It prints a cleaning report, for example *rows after cleaning: ~100*. Files created:

File	What it is
data/finance_news.db	SQLite database: tables prices, news, news_fetch_log, daily
data/daily_clean.csv	One row per trading day with all features and targets; opens in Excel
outputs/cleaning_report.txt	How many rows were dropped and why

All-in-one shortcut (same result as A + B + C):

```
python run_pipeline.py --years 0.5 --news-days 180 --delay 0.7
```

> **Be realistic about 180 days.** That is about 125 trading days, about 100 after the 20-day warm-up, so the test period is only about 20 days. Results will be very noisy: use this to learn and demonstrate the method. For a real conclusion use `--years 1 --news-days 365` or more.

Part 4 · Use each feature one by one

Every feature is a separate command. Run only what you need.

I want to...	Command	What happens
Test the install	`python tests/test_pipeline.py`	Runs 8 offline checks; no data needed
Try everything with fake data	`python run_pipeline.py --demo`	Builds a synthetic database and CSV in data/demo/
Download prices only	`python run_pipeline.py --steps prices --years 0.5`	Fills the prices table
Download news only	`python run_pipeline.py --steps news --news-days 180 --delay 0.7`	Fills the news table; resumable
Rebuild features and CSV	`python run_pipeline.py --steps build`	Recomputes daily table + CSV from stored data; use after changing keywords
Get 30 more days later	`python run_pipeline.py --steps prices,news,build --years 0.6 --news-days 210`	Adds only the missing days
Explore the data	`jupyter lab notebooks/`	Open 01_eda.ipynb: trends, distributions, keywords, correlations
Train and compare models	`jupyter lab notebooks/`	Open 02_models.ipynb: regression, Up/Down, High/Low, verdict
Look inside the database	DB Browser for SQLite (free app)	Open data/finance_news.db and browse tables
Faster, gentler news fetch	`--delay 2`	Add to the news command if you get blocked

Change what is collected

Open **config.py** in Notepad or any text editor. **SYMBOLS** lists the shares, gold, petrol and currencies (Yahoo Finance symbols); **KEYWORDS** lists the news words to count; **NEWS_QUERIES** lists the news searches. After changing SYMBOLS, re-run Step A then Step C. After changing KEYWORDS, re-run Step C only.

Optional: better sentiment scoring

Sentiment is scored when headlines are downloaded, so decide **before Step B**:

```
pip install vaderSentiment
set SENTIMENT_BACKEND=vader        (Mac/Linux: export SENTIMENT_BACKEND=vader)
```

Optional API keys (Alpha Vantage backup for currencies, NewsAPI top-up) are set the same way with `ALPHAVANTAGE_KEY` and `NEWSAPI_KEY`. They are not needed.

Part 5 · Run the notebooks and read the results

Step 5.1 · Open Jupyter

```
jupyter lab notebooks/
```

Your web browser opens. In the left panel double-click **01_eda.ipynb**. Choose the menu **Run > Run All Cells**. Wait until the star *[*]* beside the cells turns into numbers (EDA about 30 seconds).

Step 5.2 · Check you are looking at real data

The first cell prints **data source = live** for your data. If it says **SYNTHETIC DEMO DATA**, Steps A to C did not finish: the notebook falls back to the fake data when `data/daily_clean.csv` is missing.

Step 5.3 · Run the models

Open **02_models.ipynb** and choose **Run > Run All Cells** (1 to 3 minutes). The last cell prints a plain-English verdict. Charts and tables are also saved in the **outputs** folder as PNG and CSV files.

Step 5.4 · How to read the verdict

You see	It means
no reliable skill	The models did not beat the naive baseline once luck and multiple tests are accounted for. This is a valid, reportable result.
evidence of skill	A model beat the baselines on the hold-out AND walk-forward AND the Bonferroni test. Still check it on more data.
Same-day correlation high, next-day low	News mostly describes the day's move rather than predicting the next one.

Step 5.5 · Save or share a notebook

Choose **File > Save and Export Notebook As > HTML**. The HTML file opens in any browser, with all charts.

Part 6 · Fixing common problems

Problem	Fix
'python' is not recognized	Python was installed without "Add to PATH". Re-run the installer, choose Modify or Repair and tick that box; open a new terminal. On Mac use python3.
No module named ...	The environment is not active. Run the activate command from Step 1.4 and try again.
pip install fails	Check the internet connection; on a college or office network try a mobile hotspot.
A series shows 0 rows in Step A	That symbol was not found. Check the symbol on finance.yahoo.com and edit config.py. If every series is 0, you are offline or Yahoo is blocking you; wait and retry.
News stops with 'consecutive failures'	Google throttled you. Wait 10 minutes, then run the same news command with --delay 2. Progress is saved.
Very few or zero headlines for some days	Normal for weekends. Many empty days across weeks means the fetch failed: re-run Step B.
Notebook says SYNTHETIC DEMO DATA	Run Steps A, B, C first. To force demo data on purpose: set IDC409_MODE=demo.
Jupyter command not found	Environment not active, or the install in Step 1.4 did not finish. Activate it and run pip install -r requirements.txt again.
Want to start over	Delete the data folder's finance_news.db and daily_clean.csv (not the demo folder) and repeat Part 3.

Good practice: the data folder (data/demo and outputs/demo) holds **fake** data made for testing. Only quote numbers that come from your own live run, and always mention how many days of data you used.